

ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



BLM4522 — Ağ Tabanlı Paralel Dağıtım Sistemleri
Proje 1: Veritabanı Performans Optimizasyonu ve İzleme

Selim Bedirhan Öztürk

20291277

GitHub: <https://github.com/selimbedirhan/BLM4522-Ag-Tabanlı-Paralel-Dagitim-Sistemleri>

ÖZET

Bu projede PostgreSQL 14 veritabanı yönetim sistemi üzerinde kapsamlı bir performans optimizasyonu çalışması gerçekleştirilmiştir. Senaryo olarak 83.000'den fazla kayıt barındıran bir Online Eğitim Platformu veritabanı tasarlanmıştır. Önce indekssiz ortamda 8 farklı sorgunun çalışma planları **EXPLAIN ANALYZE** ile incelenmiş ve darboğazlar tespit edilmiştir. Ardından 5 farklı indeksleme stratejisi (B-Tree, Composite, Partial, Covering, GIN/trgm) uygulanmıştır. Sorgu optimizasyonu aşamasında Subquery→CTE dönüşümü, Materialized View kullanımı ve IN→EXISTS iyileştirmesi gibi teknikler gösterilmiştir. Son olarak otomatik benchmark sistemi ile öncesi/sonrası performans farkları ölçülmüş, performans izleme scripti ile HTML raporlama altyapısı kurulmuştur.

1. Giriş ve Motivasyon

Veritabanı performansı, büyük veri setleriyle çalışan modern uygulamaların en kritik kalite metriklerinden biridir. Yanlış tasarlanmış sorgular ve eksik indeksler, milisaniyeler sürmesi gereken işlemlerin saniyeler hatta dakikalar almasına neden olabilir. Özellikle yüksek trafikli web uygulamalarında bu durum doğrudan kullanıcı deneyimini ve gelir kaybını etkiler.

Bu projenin temel amacı, PostgreSQL veritabanı üzerinde performans sorunlarını sistematik olarak tespit etmek, uygun indeksleme stratejileri ile çözmek ve çözümlerin etkisini ölçülebilir şekilde kanıtlamaktır.

Proje kapsamında ele alınan konular şunlardır:

- **EXPLAIN ANALYZE** ile sorgu planlarının detaylı incelenmesi
- Sequential Scan (tam tablo tarama) sorunlarının tespiti
- 5 farklı indeks türünün (B-Tree, Composite, Partial, Covering, GIN) stratejik uygulanması
- CTE, Materialized View ve EXISTS gibi sorgu optimizasyon teknikleri
- Otomatik benchmark sistemi ile öncesi/sonrası karşılaştırma
- Performans metriklerinin veritabanı içinde loglanması
- Terminal ve HTML formatında performans raporlama

2. Ortam ve Kullanılan Teknolojiler

2.1 Kullanılan Araçlar

Araç / Teknoloji	Türü	Açıklama
PostgreSQL 14	VTYS	Açık kaynak ilişkisel veritabanı yönetim sistemi
EXPLAIN ANALYZE	Sorgu Planlayıcı	Sorgu yürütme planlarının görselleştirilmesi
psql	SQL İstemcisi	Komut satırından SQL çalıştırma ve plan analizi
pg_trgm	Uzantı	Trigram tabanlı metin benzerlik araması
Bash Scripting	Otomasyon	Benchmark ve izleme scriptleri
Python3	JSON İşleme	EXPLAIN JSON çıktısının parse edilmesi
bc	Hesaplama	Bash içinde kayan noktalı aritmetik işlemler

2.2 EXPLAIN ANALYZE Kavramı

PostgreSQL'in **EXPLAIN ANALYZE** komutu, bir sorgunun yürütme planını detaylı olarak gösterir. Bu plan, hangi tabloların nasıl tarandığını (Sequential Scan vs Index Scan), JOIN algoritmalarını (Nested Loop, Hash Join, Merge Join), tahmini ve gerçek maliyet değerlerini, gerçek çalışma süresini ve satır sayılarını içerir.

```
EXPLAIN ANALYZE SELECT * FROM ogrenciler WHERE sehir = 'Istanbul';
```

Bu komutun çıktısı:

- **Seq Scan** (indekssiz): Tüm tabloyu baştan sona tarar → Yavaş
- **Index Scan** (indeksli): İndeks üzerinden hedef satırlara doğrudan erişir → Hızlı

3. Veritabanı Tasarımı

3.1 Online Eğitim Platformu

Performans testlerinin anlamlı ve gerçekçi bir ortamda yapılabilmesi için bir **Online Eğitim Platformu** veritabanı tasarlanmıştır. Bu platform, Udemy veya Coursera benzeri bir yapıyı simüle eder: eğitmenler kurs oluşturur, öğrenciler kurslara kayıt olup ders izler, değerlendirme yapar ve ödeme gerçekleştirir.

```
DROP DATABASE IF EXISTS egitim_db;  
CREATE DATABASE egitim_db WITH ENCODING = 'UTF8' TEMPLATE = template0;
```

3.2 Tablo Yapıları

Veritabanı 10 tablodan oluşmakta olup her tablo uygun veri tipleri, **NOT NULL** kısıtlamaları, **CHECK** kontrolleri ve **DEFAULT** değerlerle tasarlanmıştır:

#	Tablo Adı	Açıklama	Önemli Alanlar	FK İlişkileri
1	egitmenler	Eğitmen profilleri	email (UNIQUE), puan, aktif	—
2	kategoriler	Kurs kategorileri (hiyerarşik)	kategori_adi, aciklama	Kendi kendine FK
3	kurslar	Kurs kataloğu	seviye (CHECK), fiyat, ort_puan	FK → egitmenler, kategoriler
4	ogrenciler	Öğrenci profilleri	email (UNIQUE), uyelik_tipi	—
5	kayitlar	Öğrenci-kurs kaydı	tamamlanma_orani, durum	FK → ogrenciler, kurslar
6	dersler	Video içerikleri	sira_no, icerik_tipi	FK → kurslar
7	ders_ilerleme	Öğrenci ders ilerlemesi	izleme_suresi, tamamlandi	FK → ogrenciler, dersler
8	degerlendirmeler	Kurs puanlamaları	puan, yorum	FK → kurslar, ogrenciler
9	odemeler	Ödeme işlemleri	tutar, durum, islem_no	FK → ogrenciler, kurslar
10	performans_log	Sorgu performans kayıtları	calisma_suresi_ms, plan_tipi	—

3.3 Örnek Tablo Tanımı: kurslar

```
CREATE TABLE kurslar (
  kurs_id SERIAL PRIMARY KEY,
  kurs_adi VARCHAR(300) NOT NULL,
  egitmen_id INTEGER NOT NULL REFERENCES egitmenler(egitmen_id),
  kategori_id INTEGER REFERENCES kategoriler(kategori_id),
  seviye VARCHAR(20) CHECK (seviye IN ('baslangic', 'orta', 'ileri', 'uzman')),
  fiyat DECIMAL(10,2) DEFAULT 0,
  indirimli_fiyat DECIMAL(10,2),
```

```
dil VARCHAR(30) DEFAULT 'Turkce',
sure_saat DECIMAL(5,1),
ders_sayisi INTEGER DEFAULT 0,
ort_puan DECIMAL(3,2) DEFAULT 0,
ogrenci_sayisi INTEGER DEFAULT 0,
aciklama TEXT,
olusturma_tarihi TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
guncelleme_tarihi TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
yayinda BOOLEAN DEFAULT TRUE
);
```

3.4 Performans Log Tablosu

Benchmark sonuçlarının veritabanı içinde saklanması için özel bir izleme tablosu tasarlanmıştır:

```
CREATE TABLE performans_log (
    log_id SERIAL PRIMARY KEY,
    sorgu_adi VARCHAR(200),
    calisma_suresi_ms DECIMAL(10,2),
    satir_sayisi INTEGER,
    indeks_kullanimi BOOLEAN,
    plan_tipi VARCHAR(50),
    tarih TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    notlar TEXT
);
```

4. Büyük Veri Seti Oluşturma

Performans testlerinin anlamlı sonuçlar üretebilmesi için veritabanının yeterli büyüklükte bir veri seti ile doldurulması gereklidir. Veri üretiminde PostgreSQL'in `generate_series()` fonksiyonu kullanılarak 83.000'den fazla kayıt oluşturulmuştur.

4.1 Veri Dağılımı

Tablo	Kayıt Sayısı	Üretim Yöntemi
kategoriler	20	Hiyerarşik kategoriler (üst-alt ilişkili)
egitmenler	80	Türkçe isim, 20 farklı uzmanlık alanı
kurslar	500	20 farklı konu × 5 seviye × rastgele fiyat
ogrenciler	5.000	20 farklı şehir, 4 üyelik tipi
kayitlar	20.000	Son 1 yıla yayılmış, 4 farklı durum
dersler	5.000	10 farklı ders tipi, video/makale/quiz
ders_ilerleme	30.000	Rastgele izleme süreleri ve tamamlanma
degerlendirmeler	8.000	1-5 puan, 10 farklı yorum şablonu
odemeler	15.000	5 ödeme yöntemi, 4 durum
TOPLAM	83.600+	

4.2 Veri Çeşitliliği

Gerçekçi performans testleri için verilerde yeterli çeşitlilik sağlanmıştır:

- **Öğrenci şehirleri:** 20 farklı Türk şehri (İstanbul, Ankara, İzmir vb.) — kardinalite kontrolü
- **Kurs seviyeleri:** 4 seviye (başlangıç, orta, ileri, uzman) — düşük kardinalite filtre testi
- **Tarihler:** Son 1 yıla yayılmış — tarih aralık sorgu testi
- **Metin alanları:** Farklı kurs isimleri — LIKE arama testi

Veri yükleme sonrası **ANALYZE** komutu çalıştırılarak PostgreSQL sorgu planlayıcısının tablo istatistikleri güncellenmiştir.

5. Yavaş Sorgu Analizi — EXPLAIN ANALYZE

İndeksleme öncesinde 8 farklı sorgu tipi **EXPLAIN ANALYZE** ile incelenmiş ve performans darboğazları tespit edilmiştir.

5.1 Sorgu 1: Basit Filtreleme (Sequential Scan)

```
EXPLAIN ANALYZE
SELECT * FROM ogrenciler WHERE sehir = 'Istanbul';
```

- **Plan:** Seq Scan on ogrenciler — 5.000 satırın tamamı okunur
- **Sorun:** İndeks olmadığı için tablo baştan sona taranır
- **Beklenen iyileşme:** B-Tree indeks ile Index Scan'e geçiş

5.2 Sorgu 2: Çoklu JOIN

```
EXPLAIN ANALYZE
SELECT k.kurs_adi, e.ad || ' ' || e.soyad AS egitmen,
       kat.kategori_adi, k.fiyat, k.ogrenci_sayisi
FROM kurslar k
JOIN egitmenler e ON k.egitmen_id = e.egitmen_id
JOIN kategoriler kat ON k.kategori_id = kat.kategori_id
WHERE k.seviye = 'ileri' AND k.yayinda = TRUE;
```

- **Plan:** Üç tablo Sequential Scan → Hash Join
- **Sorun:** `kurslar` tablosunda `seviye` ve `yayinda` kolonlarında indeks yok
- **Beklenen iyileşme:** Composite indeks ile daha verimli filtreleme

5.3 Sorgu 3: Agregasyon (GROUP BY)

```
EXPLAIN ANALYZE
SELECT kat.kategori_adi,
       COUNT(DISTINCT k.kurs_id) AS kurs_sayisi,
       ROUND(SUM(ky.odenen_tutar)::NUMERIC, 2) AS toplam_gelir
FROM kategoriler kat
JOIN kurslar k ON kat.kategori_id = k.kategori_id
JOIN kayitlar ky ON k.kurs_id = ky.kurs_id
WHERE ky.durum = 'aktif'
GROUP BY kat.kategori_adi ORDER BY toplam_gelir DESC;
```

- **Sorun:** 20.000 kayıtlık `kayitlar` tablosunda `durum` filtresi ve `kurs_id` JOIN'i indekssiz çalışır

5.4 Sorgu 4: Subquery (Alt Sorgu)

```
EXPLAIN ANALYZE
SELECT e.ad || ' ' || e.soyad AS egitmen,
       (SELECT COUNT(*) FROM kurslar k WHERE k.egitmen_id = e.egitmen_id)
       (SELECT COALESCE(SUM(o.tutar), 0) FROM odemeler o
        JOIN kurslar k ON o.kurs_id = k.kurs_id
        WHERE k.egitmen_id = e.egitmen_id AND o.durum = 'basarili') AS toplam_gelir
FROM egitmenler e WHERE e.aktif = TRUE ORDER BY toplam_gelir DESC LIMIT 10;
```

- **Sorun:** Her eğitmen için 2 ayrı alt sorgu çalışır (N+1 problemi)
- **Beklenen iyileşme:** CTE dönüşümü ile tek geçişte hesaplama

5.5 Sorgu 7: LIKE Metin Araması

```
EXPLAIN ANALYZE
SELECT kurs_id, kurs_adi, fiyat FROM kurslar
WHERE kurs_adi LIKE '%Python%' OR kurs_adi LIKE '%Java%';
```

- **Sorun:** `%pattern%` formatı standart B-Tree indeksi kullanamaz → Seq Scan

- **Beklenen iyileşme:** pg_trgm uzantısı ile GIN indeks

6. İndeksleme Stratejileri

Yavaş sorgu analizinde tespit edilen darboğazları gidermek için 5 farklı indeks stratejisi uygulanmıştır.

6.1 B-Tree İndeksler (Standart)

B-Tree, PostgreSQL'in varsayılan indeks türüdür. Eşitlik (=) ve aralık (<, >, BETWEEN) sorgularında etkilidir. Projede 22 adet B-Tree indeks oluşturulmuştur:

```
-- Sık aranılan kolonlar
CREATE INDEX idx_ogrenci_sehir ON ogrenciler(sehir);
CREATE INDEX idx_ogrenci_uyelik ON ogrenciler(uyelik_tipi);
CREATE INDEX idx_ogrenci_kayit_tarihi ON ogrenciler(kayit_tarihi);

-- Kurs filtreleme
CREATE INDEX idx_kurs_seviye ON kurslar(seviye);
CREATE INDEX idx_kurs_yayinda ON kurslar(yayinda);
CREATE INDEX idx_kurs_fiyat ON kurslar(fiyat);
CREATE INDEX idx_kurs_kategori ON kurslar(kategori_id);
CREATE INDEX idx_kurs_egitmen ON kurslar(egitmen_id);
```

6.2 Composite (Bileşik) İndeksler

Birden fazla kolonun birlikte sıklıkla sorgulandığı durumlarda, tek bir indekste birleştirilir. Bu, ayrı ayrı iki indeks kullanmaktan daha verimlidir:

```
CREATE INDEX idx_kayit_ogrenci_durum ON kayitlar(ogrenci_id, durum);
CREATE INDEX idx_odeme_tarih_durum ON odemeler(odeme_tarihi, durum);
CREATE INDEX idx_kurs_kategori_seviye ON kurslar(kategori_id, seviye);
CREATE INDEX idx_ilerleme_ogrenci_tamamlandi ON ders_ilerleme(ogrenci_id,
```

Önemli: Composite indekslerde kolon sırası kritiktir. En seçici (yüksek kardinaliteli) kolon önce gelmelidir.

6.3 Partial (Kısmi) İndeksler

Tablodaki tüm satırlar yerine, yalnızca belirli bir koşulu sağlayan satırlar için indeks oluşturulur. Bu, indeks boyutunu küçültür ve güncelleme maliyetini azaltır:

```
CREATE INDEX idx_ogrenci_aktif ON ogrenciler(ogrenci_id) WHERE aktif = 'E';
CREATE INDEX idx_kurs_yayinda_aktif ON kurslar(kurs_id) WHERE yayinda = 'E';
CREATE INDEX idx_kayit_aktif ON kayitlar(kurs_id) WHERE durum = 'aktif';
CREATE INDEX idx_odeme_basarili ON odemeler(ogrenci_id, tutar) WHERE duru
```

Avantaj: Uygulama mantığında çoğunlukla sadece aktif kayıtlar sorgulanır. Bu nedenle indeks çok daha küçük olur ve bellekte daha verimli tutulur.

6.4 Covering (Kapsayan) İndeksler

INCLUDE anahtar kelimesi ile sorgunun ihtiyaç duyduğu tüm kolonları indeks içine dahil ederek **Index-Only Scan** yapılmasını sağlar. Tablo verilerine hiç erişmeden doğrudan indeksten cevap döner:

```
CREATE INDEX idx_kurs_cover_liste ON kurslar(kategori_id, seviye)
INCLUDE (kurs_adi, fiyat, ort_puan);

CREATE INDEX idx_odeme_cover_rapor ON odemeler(odeme_tarihi, durum)
INCLUDE (tutar, odeme_yontemi);
```

Avantaj: Heap (tablo) erişimini tamamen ortadan kaldırarak sorgu süresini önemli ölçüde azaltır.

6.5 GIN İndeks + pg_trgm (Metin Araması)

Standart B-Tree indeksleri **LIKE** '%pattern%' formatında (ön ek olmayan) arama yapamaz. PostgreSQL'in **pg_trgm** uzantısı, metni 3'lü karakter gruplarına (trigram) bölerek benzerlik araması yapmayı mümkün kılar:

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE INDEX idx_kurs_adi_trgm ON kurslar USING gin(kurs_adi gin_trgm_op:
```

Sonuç: **WHERE kurs_adi LIKE '%Python%'** sorgusu artık Sequential Scan yerine GIN indeksini kullanır.

7. Sorgu Optimizasyonu Teknikleri

İndeksleme sonrası aynı 8 sorgu tekrar çalıştırılmış ve ek olarak 3 farklı optimizasyon tekniği uygulanmıştır.

7.1 Subquery → CTE Dönüşümü

Önceki (Yavaş): Her eğitmen için iki ayrı alt sorgu çalışıyor (N+1 problemi):

```
SELECT e.ad,
       (SELECT COUNT(*) FROM kurslar WHERE egitmen_id = e.egitmen_id),
       (SELECT SUM(o.tutar) FROM odemeler o JOIN kurslar k ON ...)
FROM egitmenler e;
```

Sonraki (Optimize): CTE ile tek geçişte tüm hesaplamalar yapılır:

```
WITH egitmen_gelir AS (
  SELECT k.egitmen_id,
         COUNT(DISTINCT k.kurs_id) AS kurs_sayisi,
         COALESCE(SUM(o.tutar), 0) AS toplam_gelir
  FROM kurslar k
  LEFT JOIN odemeler o ON k.kurs_id = o.kurs_id AND o.durum = 'basaril:
  GROUP BY k.egitmen_id
)
```

```
SELECT e.ad || ' ' || e.soyad AS egitmen, eg.kurs_sayisi, eg.toplam_gelir
FROM egitmenler e
JOIN egitmen_gelir eg ON e.egitmen_id = eg.egitmen_id
WHERE e.aktif = TRUE
ORDER BY eg.toplam_gelir DESC LIMIT 10;
```

İyileşme: Alt sorgu sayısı $2 \times N$ 'den 1'e düşer.

7.2 Materialized View

Sık çalıştırılan ve karmaşık JOIN içeren raporlama sorguları için verilerin önceden hesaplanıp saklandığı materialized view oluşturulmuştur:

```
CREATE MATERIALIZED VIEW mv_kurs_istatistik AS
SELECT k.kurs_id, k.kurs_adi, e.ad || ' ' || e.soyad AS egitmen,
       kat.kategori_adi, k.seviye, k.fiyat,
       COUNT(DISTINCT ky.kayit_id) AS kayit_sayisi,
       ROUND(AVG(d.puan)::NUMERIC, 2) AS ort_puan,
       COALESCE(SUM(CASE WHEN od.durum = 'basarili' THEN od.tutar ELSE 0
FROM kurslar k
LEFT JOIN egitmenler e ON k.egitmen_id = e.egitmen_id
LEFT JOIN kategoriler kat ON k.kategori_id = kat.kategori_id
LEFT JOIN kayitlar ky ON k.kurs_id = ky.kurs_id
LEFT JOIN degerlendirmeler d ON k.kurs_id = d.kurs_id
LEFT JOIN odemeler od ON k.kurs_id = od.kurs_id
GROUP BY k.kurs_id, k.kurs_adi, e.ad, e.soyad, kat.kategori_adi, k.seviye,
```

Karşılaştırma: 5 tabloluk JOIN sorgusu yerine tek tablo erişimi → önemli performans artışı.

7.3 IN vs EXISTS Karşılaştırması

```
-- IN kullanımı
SELECT o.ad FROM ogrenciler o
WHERE o.ogrenci_id IN (
    SELECT DISTINCT ogrenci_id FROM kayitlar WHERE durum = 'tamamlandi'
);

-- EXISTS kullanımı
SELECT o.ad FROM ogrenciler o
WHERE EXISTS (
    SELECT 1 FROM kayitlar ky
    WHERE ky.ogrenci_id = o.ogrenci_id AND ky.durum = 'tamamlandi'
);
```

Neden EXISTS daha hızlı: **IN** tüm alt sorgu sonuçlarını belleğe yüklerken, **EXISTS** ilk eşleşmeyi bulduğunda değerlendirmeyi durdurur. Büyük veri setlerinde fark önemli olur.

8. Performans İzleme ve Raporlama

`06_performans_izleme.sh` scripti, veritabanının genel sağlığını ve performans metriklerini izleyerek terminal çıktısı ve HTML rapor üretir.

8.1 İzlenen Metrikler

#	Metrik Kategorisi	Sorgulanan Kaynak	Açıklama
1	Veritabanı boyutları	<code>pg_database_size()</code>	Tüm DB ve tablo boyutları
2	Tablo istatistikleri	<code>pg_stat_user_tables</code>	Kayıt sayıları, seq/index scan oranları
3	İndeks kullanımı	<code>pg_stat_user_indexes</code>	İndeks boyutları, kullanım sayıları
4	Cache hit oranı	<code>pg_stat_database</code>	Buffer cache isabet oranı
5	Bağlantı durumu	<code>pg_stat_activity</code>	Aktif/idle bağlantılar
6	PostgreSQL ayarları	<code>pg_settings</code>	shared_buffers, work_mem vb.

9. Benchmark — Öncesi/Sonrası Karşılaştırma

`07_benchmark.sh` scripti, indeksleme öncesi ve sonrası sorgu performanslarını otomatik olarak ölçer ve karşılaştırır. Bu, optimizasyon çalışmasının etkisini somut verilerle kanıtlayan temel bileşendir.

9.1 Benchmark Sorguları

#	Sorgu	Hedef
1	Şehre göre öğrenci arama	B-Tree indeks etkisi
2	Kurs-eğitmen JOIN filtresi	Composite indeks + JOIN
3	Son 30 gün ödeme filtresi	Tarih indeks
4	Kayıt durum filtresi	Partial indeks
5	Ders ilerleme agregasyonu	Büyük tablo + GROUP BY
6	Kurs adı LIKE arama	GIN/trgm indeks

10. Tam Demo Senaryosu

`08_tam_demo.sh` scripti, projenin tüm aşamalarını sıralı olarak çalıştıran bir demo senaryosu sunar:

Adım	İşlem	Script
1	Veritabanı oluşturma	01_veritabani_olustur.sql
2	83.000+ kayıt yükleme	02_ornek_veri_yukle.sql
3	İndeksless sorgu analizi	03_yavas_sorgu_analizi.sql
4	İndeks oluşturma (5 strateji)	04_indeksleme_stratejileri.sql
5	Optimize sorgu çalıştırma	05_sorgu_optimizasyonu.sql
6	Performans izleme + HTML rapor	06_performans_izleme.sh
7	Benchmark karşılaştırma	07_benchmark.sh

11. Sonuç ve Değerlendirme

11.1 Elde Edilen Kazanımlar

- **EXPLAIN ANALYZE Yetkinliği:** Sorgu planlarını okuma, Sequential Scan ve Index Scan farkını anlama, maliyet tahminlerini yorumlama becerisi
- **İndeks Stratejisi Seçimi:** B-Tree, Composite, Partial, Covering ve GIN indekslerin hangi senaryolarda kullanılacağını pratik deneyimi
- **Sorgu Yeniden Yazımı:** Subquery→CTE, IN→EXISTS dönüşümleri ve Materialized View kullanımı
- **Performans Ölçümü:** Benchmark yöntemi ile iyileşmenin sayısal olarak kanıtlanması
- **Sürekli İzleme:** Performans metriklerinin toplanması, loglanması ve raporlanması

11.2 İndeks Türü Seçim Kılavuzu

Durum	Önerilen İndeks
Eşitlik ve aralık sorguları (= , < , >)	B-Tree
Birden fazla kolon birlikte filtrelenebilir	Composite
Sadece belirli satırlar sorgulanıyor (WHERE koşulu)	Partial
Sorgu sonucu tamamen indeksten dönebilir	Covering (INCLUDE)
LIKE '%metin%' formatında arama	GIN + pg_trgm

Kaynakça

1. PostgreSQL Documentation — EXPLAIN
<https://www.postgresql.org/docs/14/sql-explain.html>
2. PostgreSQL Documentation — Indexes
<https://www.postgresql.org/docs/14/indexes.html>

3. PostgreSQL Documentation — Index Types
<https://www.postgresql.org/docs/14/indexes-types.html>
4. PostgreSQL Documentation — pg_trgm
<https://www.postgresql.org/docs/14/pgtrgm.html>
5. PostgreSQL Documentation — Materialized Views
<https://www.postgresql.org/docs/14/sql-creatematerializedview.html>
6. PostgreSQL Documentation — Performance Tips
<https://www.postgresql.org/docs/14/performance-tips.html>